

Experiment No : 3

Title :Perform Bubble Sort, Selection Sort, Insertion Sort, Quick Sort, and Merge Sort using C

Name of Student:Shivam Sevewar

Class: E&TC

Div: SY1

PRN No:B25ET1042

Batch:B

Program:

```
#include <stdio.h>
```

```
void swap(int *a, int *b)
```

```
{  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

```
void bubbleSort(int arr[], int n)
```

```
{  
    int i, j;  
  
    for(i = 0; i < n - 1; i++)  
        for(j = 0; j < n - i - 1; j++)  
            if(arr[j] > arr[j + 1])  
                swap(&arr[j], &arr[j + 1]);  
}
```

```
void selectionSort(int arr[], int n)
```

```
{  
    int i, j, min;  
  
    for(i = 0; i < n - 1; i++)  
    {  
        min = i;
```

```

        for(j = i + 1; j < n; j++)
            if(arr[j] < arr[min])
                min = j;

        if(min != i)
            swap(&arr[i], &arr[min]);
    }
}

```

```

void insertionSort(int arr[], int n)
{
    int i, j, key;

    for(i = 1; i < n; i++)
    {
        key = arr[i];
        j = i - 1;

        while(j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = key;
    }
}

```

```

int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;
    int j;

    for(j = low; j < high; j++)

```

```

    {
        if(arr[j] < pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }

    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high)
{
    int pi;

    if(low < high)
    {
        pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

void merge(int arr[], int low, int mid, int high)
{
    int i = low, j = mid + 1, k = low;
    int b[100];

    while(i <= mid && j <= high)
    {
        if(arr[i] <= arr[j])
            b[k++] = arr[i++];
        else
            b[k++] = arr[j++];
    }
}

```

```

    }

    while(i <= mid)
        b[k++] = arr[i++];

    while(j <= high)
        b[k++] = arr[j++];

    for(i = low; i <= high; i++)
        arr[i] = b[i];
}

void mergeSort(int arr[], int low, int high)
{
    int mid;

    if(low < high)
    {
        mid = (low + high) / 2;
        mergeSort(arr, low, mid);
        mergeSort(arr, mid + 1, high);
        merge(arr, low, mid, high);
    }
}

void printArray(int arr[], int n)
{
    int i;

    for(i = 0; i < n; i++)
        printf("%d ", arr[i]);

    printf("\n");
}

```

```
int main()
{
    int arr[100], n, choice;
    int i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);

    for(i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    printf("\n1. Bubble Sort");
    printf("\n2. Selection Sort");
    printf("\n3. Insertion Sort");
    printf("\n4. Quick Sort");
    printf("\n5. Merge Sort");

    printf("\nEnter your choice: ");
    scanf("%d", &choice);

    switch(choice)
    {
        case 1:
            bubbleSort(arr, n);
            break;

        case 2:
            selectionSort(arr, n);
            break;

        case 3:
            insertionSort(arr, n);
            break;
```

```
        case 4:
            quickSort(arr, 0, n - 1);
            break;

        case 5:
            mergeSort(arr, 0, n - 1);
            break;

        default:
            printf("Invalid choice!");
            return 0;
    }

    printf("\nSorted array: ");
    printArray(arr, n);

    return 0;
}
```

Output:

Output

```
Enter number of elements: 4
Enter 4 elements:
40
010
45
80
```

```
1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Quick Sort
5. Merge Sort
Enter your choice: 2
```

```
Sorted array: 10 40 45 80
```

```
=== Code Execution Successful ===
```

